Reference Model API
C++ Class · extern "C" · Python

LonghornSilicon ACU — Blocks 1 & 2

Date: 2026-05-14
Covers: Precision Controller, MAC Array
Languages: C++17, C99, Python 3.10+
Build: `make` in `sw/reference_model/`

Purpose. Formal API reference for the bit-accurate reference implementations of the LonghornSilicon ACU blocks. Each block ships three language bindings (C++ class, extern "C", Python); all three pass identical test vectors. A compiler backend chooses whichever binding fits its build system; numerical results are guaranteed identical.

Contents

1. Quick Start

```
cd sw/reference_model
make test-all      # build C++ tests, run them, run all Python tests
make integration   # end-to-end attention-tile worked example
```

Requires Python 3.10+, NumPy, and a C++17 compiler.

2. Precision Controller

2.1 C++ Class

```
#include "precision_controller_ref.hpp"

lhsi::PrecisionController pc;
std::vector<int32_t> scores(/* 4096 INT8 values */);
bool decision = pc.process_tile(scores);   // true = FP16, false = INT8
```

2.2 extern "C" API

```
#include "precision_controller_ref.hpp"

lhsi_pc_handle_t* h = lhsi_pc_create();
int32_t scores[4096] = { /* ... */ };
int decision = lhsi_pc_process_tile(h, scores, 4096);
lhsi_pc_destroy(h);
```

Stateless variant (no handle):

```
int decision = lhsi_pc_decide(scores, 4096);
```

2.3 Python

```
from precision_controller_ref import PrecisionController

pc = PrecisionController()
decision = pc.process_tile(scores)
# Or stateless:
decision = PrecisionController.decide(scores)
```

2.4 Streaming API

For driver code or cycle-accurate simulators that want to issue one score per clock (mirrors the chip's AXI-Stream interface).

```
pc.reset();
for (size_t i = 0; i < scores.size(); ++i) {
    pc.tick(/*s_valid=*/true, scores[i], /*s_last=*/(i == scores.size()
        - 1));
    if (pc.d_valid()) {
        bool decision = pc.d_fp16();
```

```
    }
}
```

The Python class has the same shape: `pc.tick(s_valid=True, s_data=x, s_last=False)`.

3. MAC Array

3.1 C++ Class

```cpp
#include "mac_array_ref.hpp"

lhsi::mac::MacArray mac;

// INT8 path
std::vector<int8_t> A(M*K), B(K*N);
std::vector<int32_t> C(M*N);
mac.matmul_int8(A.data(), B.data(), C.data(), M, K, N);

// FP16 path (float storage; fp16 rounding on output)
std::vector<float> Af(M*K), Bf(K*N), Cf(M*N);
mac.matmul_fp16(Af.data(), Bf.data(), Cf.data(), M, K, N);

// Cost estimate for the compiler's scheduler
auto cost = mac.estimate(M, K, N, lhsi::mac::MacArray::DType::Int8);
// cost.cycles, cost.energy_pj
```

3.2 extern "C" API

```cpp
#include "mac_array_ref.hpp"

int8_t  A[M*K], B[K*N];
int32_t C[M*N];
lhsi_mac_matmul_int8(A, B, C, M, K, N);

float Af[M*K], Bf[K*N], Cf[M*N];
lhsi_mac_matmul_fp16(Af, Bf, Cf, M, K, N);

lhsi_mac_cost_t cost;
lhsi_mac_estimate(M, K, N, LHSI_MAC_DTYPE_INT8, &cost);
```

3.3 Python

```python
import numpy as np
from mac_array_ref import MacArray

mac = MacArray()

# INT8
A = np.random.randint(-128, 128, (M, K), dtype=np.int8)
B = np.random.randint(-128, 128, (K, N), dtype=np.int8)
C = mac.matmul_int8(A, B)          # numpy int32 array
```

```
# FP16
A = np.random.uniform(-1, 1, (M, K)).astype(np.float16).astype(np.
    float32)
B = np.random.uniform(-1, 1, (K, N)).astype(np.float16).astype(np.
    float32)
C = mac.matmul_fp16(A, B)         # float32 storage, fp16-rounded output

# Cost
cost = mac.estimate(M, K, N, dtype="int8")
print(cost.cycles, cost.energy_pj)
```

3.4 FP16 Utility Helpers

Exposed for boundaries where you need explicit fp16 rounding without going through a matmul.

```
float    rounded = lhsi::mac::round_to_fp16(x);
uint16_t bits    = lhsi::mac::float_to_fp16_bits(x);
float    back    = lhsi::mac::fp16_bits_to_float(bits);
```

Same functions are available in Python:

```
from mac_array_ref import (
    round_to_fp16, float_to_fp16_bits, fp16_bits_to_float,
)
```

4. Numerical Semantics (Frozen)

4.1 Precision Controller

Pure integer arithmetic in unsigned fixed-width modular form:

- Input `s_data`: `SCORE_WIDTH`-bit signed two's complement.
- `max_acc`: `SCORE_WIDTH` bits (unsigned absolute).
- `sum_acc`: `SCORE_WIDTH` $+ \log_2(N)$ bits (unsigned).
- LHS = max $\ll \log_2(N)$ (free shift; wire routing).
- RHS = (sum $\ll 3$) + (sum $\ll 1$) = sum $\times 10$.
- Decision: LHS $>$ RHS $\Rightarrow$ FP16.

If the reference disagrees with the chip on any input, the reference is wrong. File an issue with the failing tile from `rtl/tb/testvectors/`.

4.2 MAC Array, INT8 Path

Bit-exact integer matmul, `int32_t` accumulator, no saturation. The compiler is responsible for any requantization after the matmul.

4.3 MAC Array, FP16 Path

`float` internally; IEEE-754 binary16 round-to-nearest-even on each output element. See `docs/mac_array_design.pdf` §3.2 for the v0.1 simplifications and what may change in v0.2.

5. Linking Options

5.1 Static Library

```
make static    # produces libprecision_controller_ref.a +
   libmac_array_ref.a
```

Then link your compiler binary:

```
g++ my_compiler.cpp -lprecision_controller_ref -lmac_array_ref \
    -L /path/to/sw/reference_model -o my_compiler
```

5.2 Shared Library / FFI

```
make shared    # produces *.so for FFI from any language
```

Python's `ctypes` or Rust's `libloading` can dlopen the `.so` and call any `lhsi_*` function declared in `precision_controller_ref.hpp` or `mac_array_ref.hpp`.

5.3 Direct Source Inclusion

```
g++ my_compiler.cpp \
    sw/reference_model/precision_controller_ref.cpp \
    sw/reference_model/mac_array_ref.cpp \
    -I sw/reference_model -o my_compiler
```

The reference `.cpp` files are small (∼300 lines combined) so direct inclusion is also fine.

6. End-to-End Example

The cleanest pointer for a compiler engineer: run and read `sw/reference_model/integration_example.py`. It composes both blocks the way a compiler backend should — process a $QK^\top$ tile, push to the precision controller, dispatch to either the INT8 or FP16 MAC path, accumulate.

```
make integration
```

Sample output (100 synthetic attention tiles, 21% expected FP16):

```
Routed:  INT8 = 82, FP16 = 18
Truth :  INT8 = 82, FP16 = 18

Per-tile cost:
  INT8:   1040 cycles, 131.1 nJ
  FP16:   4112 cycles, 655.4 nJ

Mixed-precision policy:       159k cycles, 22 uJ
All-FP16 baseline:            411k cycles, 66 uJ
Speedup:        2.58x
Energy saving:  65.6%
```

The routing matches ground truth exactly (82/18 vs 82/18); the cost model shows the win from mixed-precision dispatch.

7. Three Compiler-Use Scenarios

`sw/reference_model/example_compiler_use.py` shows three levels of sophistication for a compiler backend using just the precision controller:

1. **Level A — Naive per-tile dispatch.** For each tile, ask the gate and emit either an INT8 or FP16 kernel call. Simplest possible integration.
2. **Level B — Batched index lists.** Collect decisions for a window of tiles, then batch all INT8 tiles to one kernel launch and all FP16 tiles to another. Amortizes per-tile overhead.
3. **Level C — Offline calibration.** Run the gate over a calibration set offline, observe that some workloads are overwhelmingly one precision, and emit a workload-specific policy that skips the runtime gate.

The end-to-end `integration_example.py` (§6) is the next level up, composing both blocks.

8. Verification Status

Table 1: Reference-model verification status.

Block	Test	Status
Precision Controller	143/143 RTL replay tiles, bit-exact (C++ and Py)	pass
Precision Controller	Stateful streaming = batched = stateless	pass
Precision Controller	extern "C" = C++ class	pass
Precision Controller	Canonical edges (spike = 10 / 11)	pass
Precision Controller	Accumulator reset between tiles	pass
MAC Array	INT8 matmul vs. naive int64 reference	pass (exact)
MAC Array	FP16 matmul vs. naive double reference	pass (within $5 \cdot 10^{-3}$)
MAC Array	Edge cases (zero, identity, signed extremes)	pass
MAC Array	extern "C" = C++ class	pass
MAC Array	FP16 round-trip helpers	pass
MAC Array	Cost-estimate sanity	pass
End-to-end	Integration example routing matches ground truth	82/18 vs 82/18

9. Where to Find Each File

```
sw/reference_model/
  precision_controller_ref.hpp            -- block 1 header
  precision_controller_ref.cpp            -- block 1 implementation
  precision_controller_ref.py             -- block 1 Python ref
  test_precision_controller_ref.cpp       -- block 1 C++ tests
  test_precision_controller_ref.py        -- block 1 Python tests

  mac_array_ref.hpp                       -- block 2 header
  mac_array_ref.cpp                       -- block 2 implementation
  mac_array_ref.py                        -- block 2 Python ref
  test_mac_array_ref.cpp                  -- block 2 C++ tests
  test_mac_array_ref.py                   -- block 2 Python tests

  integration_example.py                  -- composes both blocks
  example_compiler_use.py                 -- three sophistication levels

  Makefile                                -- build / test / shared /
     static
  README.md                               -- markdown form of this doc

docs/
  isa/precision_controller_isa.pdf        -- ISA spec for block 1
  mac_array_design.pdf                    -- design rationale for block 2
  sw_overview.pdf                         -- compiler-team entry point
  reference_model_api.pdf                 -- THIS DOCUMENT
```

This document is the canonical API reference for the LonghornSilicon ACU reference models. Updated as each new block lands in software.